

Asignación IV informe de investigación

Daniel Garrido

UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

Lenguaje y compiladores 2026-I

Félix Márquez

12 de julio de 2026

Introducción

El análisis léxico constituye la primera etapa del proceso de compilación de un lenguaje de programación. Su función principal consiste en leer el programa fuente carácter por carácter para reconocer los componentes léxicos o **tokens**, tales como palabras reservadas, identificadores, operadores, números y símbolos. Este proceso, conocido como **tokenización**, permite detectar errores léxicos antes de continuar con las siguientes fases del compilador, como el análisis sintáctico y el análisis semántico.

El reconocimiento de estos componentes se fundamenta en el uso de **autómatas finitos** y **expresiones regulares**, herramientas que permiten describir y validar los patrones que conforman un lenguaje formal. Asimismo, para lenguajes con estructuras más complejas, como aquellos definidos mediante gramáticas libres de contexto, resulta necesario el uso de **autómatas de pila**, los cuales poseen una mayor capacidad de reconocimiento.

En el presente trabajo se desarrollan los conceptos fundamentales relacionados con el análisis léxico y su aplicación práctica. Para ello, se estudian los autómatas de pila y su importancia dentro de la teoría de compiladores; posteriormente se implementa un analizador léxico para archivos **Dockerfile** utilizando expresiones regulares y, finalmente, se construye un segundo analizador léxico para un subconjunto del lenguaje **Rust** empleando el metacompilador **Flex**. Como complemento, se presenta una breve investigación sobre la aplicación de Flex en el área de la seguridad informática.

El objetivo de esta actividad es comprender el funcionamiento de los analizadores léxicos, aplicar los conceptos estudiados durante la asignatura y desarrollar herramientas capaces de reconocer los componentes léxicos de un lenguaje formal.

Autómatas de Pila

Un **Autómata de Pila (AP)** es un modelo computacional utilizado para reconocer **lenguajes libres de contexto**, los cuales poseen estructuras más complejas que los lenguajes regulares. A diferencia de los autómatas finitos, un autómata de pila dispone de una **memoria adicional denominada pila**, la cual le permite almacenar información temporal durante el procesamiento de una cadena.

Esta característica hace posible reconocer estructuras anidadas o balanceadas, como paréntesis, llaves y bloques de código, que no pueden ser reconocidas únicamente mediante un autómata finito.

Componentes de un Autómata de Pila

Un Autómata de Pila está conformado por los siguientes elementos:

- **Q:** conjunto finito de estados.
- **Σ :** alfabeto de entrada.
- **Γ :** alfabeto de la pila.
- **δ :** función de transición.
- **q_0 :** estado inicial.
- **Z_0 :** símbolo inicial de la pila.
- **F:** conjunto de estados de aceptación.

Durante el procesamiento de una cadena, el autómata puede realizar tres operaciones básicas sobre la pila:

- Insertar un símbolo (*Push*).
- Retirar un símbolo (*Pop*).
- Consultar el símbolo ubicado en la cima de la pila.

Funcionamiento

El funcionamiento de un Autómata de Pila consiste en leer la cadena de entrada símbolo por símbolo mientras utiliza la pila para almacenar información que será necesaria posteriormente.

Cuando el lenguaje presenta estructuras que deben abrirse y cerrarse correctamente, como los paréntesis, el autómata inserta un símbolo en la pila al encontrar un paréntesis de apertura y lo elimina cuando encuentra el correspondiente paréntesis de cierre. Si al finalizar la lectura la pila queda vacía y el autómata alcanza un estado de aceptación, la cadena pertenece al lenguaje; de lo contrario, será rechazada.

Ejemplo

Considere el siguiente lenguaje:

$$L = \{ ({}^n) {}^n \mid n \geq 1 \}$$

Este lenguaje acepta cadenas como:

()

(())

((()))

Pero rechaza cadenas como:

((

)

(())

Tabla 1

Durante el procesamiento de la cadena ((())), el autómata realiza las siguientes operaciones:

Símbolo leído	Operación sobre la pila
(	Push
(	Push
(	Push
)	Pop
)	Pop
)	Pop

Nota. Al finalizar la lectura, la pila queda vacía, por lo que la cadena es aceptada.

Comparación con los Autómatas Finitos

Los Autómatas Finitos Determinísticos (AFD) y No Determinísticos (AFND) únicamente pueden reconocer **lenguajes regulares**, ya que no poseen memoria para almacenar información durante la ejecución.

En cambio, los Autómatas de Pila incorporan una pila como memoria auxiliar, permitiéndoles reconocer lenguajes libres de contexto, los cuales describen estructuras propias de los lenguajes de programación, como expresiones aritméticas, bloques de instrucciones y sentencias anidadas.

Por esta razón, los Autómatas de Pila poseen un mayor poder de reconocimiento que los AFD y AFND.

Importancia de los Autómatas de Pila

Los Autómatas de Pila desempeñan un papel fundamental en el desarrollo de compiladores, ya que permiten verificar la estructura sintáctica de un programa. Gracias a su capacidad para manejar información mediante una pila, es posible validar correctamente expresiones con paréntesis, bloques delimitados por llaves, estructuras condicionales y ciclos, elementos que forman parte de la mayoría de los lenguajes de programación modernos.

Los Autómatas de Pila representan una herramienta fundamental dentro de la teoría de compiladores debido a su capacidad para reconocer lenguajes libres de contexto. A diferencia de los autómatas finitos, incorporan una memoria en forma de pila que les permite procesar estructuras anidadas y balanceadas, ampliamente utilizadas en los lenguajes de programación. Por esta razón, constituyen la base teórica de los analizadores sintácticos y complementan el trabajo realizado previamente por el analizador léxico durante el proceso de compilación.

Implementación de un Lexer para Docker mediante Expresiones Regulares

Docker es una plataforma que permite crear, distribuir y ejecutar aplicaciones mediante contenedores. La construcción de una imagen Docker se realiza a través de un archivo denominado **Dockerfile**, el cual contiene un conjunto de instrucciones que describen paso a paso cómo construir dicha imagen.

Cada línea del archivo está formada por instrucciones conocidas como **directivas**, acompañadas de los parámetros necesarios para su ejecución. Entre las directivas más utilizadas se encuentran **FROM**, **RUN**, **COPY**, **CMD**, **WORKDIR**, **ENV** y **EXPOSE**, además de comentarios y diferentes tipos de valores, como rutas, nombres de imágenes y cadenas de texto.

Para esta actividad se definió un lenguaje **L**, correspondiente a un subconjunto del lenguaje utilizado en los archivos Dockerfile. El objetivo del analizador léxico consiste en reconocer los principales componentes léxicos presentes en este tipo de archivos mediante el uso de expresiones regulares.

Tabla 2

Los tokens definidos para este lenguaje se muestran en la siguiente tabla.

Token	Descripcion
FROM	Imagen base
RUN	Ejecución de comandos
COPY	Copiar archivos
CMD	Comando por defecto
WORKDIR	Directorio de trabajo
ENV	Variables de entorno
EXPOSE	Puertos expuestos
IMAGE	Nombre de imagen Docker
PATH	Rutas del sistema
STRING	Cadenas de texto
COMMENT	Comentarios
NEWLINE	Fin de línea
SKIP	Espacios y tabulaciones

*Nota. Se definió un lenguaje **L**, correspondiente a un subconjunto del lenguaje utilizado en los archivos Dockerfile.*

Implementación

El analizador léxico fue desarrollado en **Python** utilizando el módulo **re**, el cual permite trabajar con expresiones regulares para reconocer patrones dentro de una cadena de caracteres.

Cada componente léxico del lenguaje fue representado mediante una expresión regular asociada a un token. Durante la ejecución, el programa lee el archivo Dockerfile y compara cada fragmento de texto con las expresiones definidas. Cuando encuentra una coincidencia válida, genera el token correspondiente junto con su lexema.

Tabla 3

Las expresiones regulares utilizadas se presentan en la siguiente tabla.

Token	Expresión regular
FROM	\bFROM\b
RUN	\bRUN\b
COPY	\bCOPY\b
CMD	\bCMD\b
WORKDIR	\bWORKDIR\b
ENV	\bENV\b
EXPOSE	\bEXPOSE\b
IMAGE	[a-zA-Z0-9._/-]+(:[a-zA-Z0-9._-]+)?
PATH	(/[a-zA-Z0-9._-]+)+
STRING	"[^"]*"
COMMENT	#. *
NEWLINE	\n
SKIP	[\t]+

Nota. Cada componente léxico del lenguaje fue representado mediante una expresión regular asociada a un token.

Explicación del código

El programa comienza definiendo una lista de tokens junto con la expresión regular asociada a cada uno de ellos. Posteriormente, todas las expresiones son combinadas para construir un único patrón de búsqueda.

Durante la lectura del archivo, el analizador identifica el tipo de token encontrado y obtiene el lexema correspondiente. Los espacios en blanco, tabulaciones y comentarios son ignorados, mientras que las instrucciones y valores válidos son reportados como parte de la salida del programa.

Cuando un fragmento de texto no coincide con ninguna de las expresiones definidas, el analizador genera un error léxico, indicando el símbolo que no pudo ser reconocido.

Ejecución

Para verificar el funcionamiento del analizador se utilizó el siguiente archivo Dockerfile como entrada:

```
FROM python:3.12
```

```
WORKDIR /app
```

```
COPY . /app
```

```
RUN pip install flask
```

```
EXPOSE 8000
```

```
CMD ["python", "app.py"]
```

La salida obtenida corresponde a la identificación de cada uno de los tokens definidos para el lenguaje, mostrando el tipo de token y el lexema reconocido.

Figura 1

Salida del analizador léxico para el primer ejemplo de Dockerfile, donde se identifican correctamente los tokens y lexemas reconocidos durante el proceso de tokenización.

```
=====
EJEMPLO 1: Dockerfile válido
=====
```

```
FROM python:3.12
WORKDIR /app
COPY . /app
RUN pip install flask
EXPOSE 8000
CMD ["python", "app.py"]
```

TOKEN	LEXEMA	LÍNEA	COLUMNA
FROM	FROM	1	1
IMAGE	python:3.12	1	6
WORKDIR	WORKDIR	2	1
PATH	/app	2	9
COPY	COPY	3	1
WORD	.	3	6
PATH	/app	3	8
RUN	RUN	4	1
WORD	pip	4	5
WORD	install	4	9
WORD	flask	4	17
EXPOSE	EXPOSE	5	1
NUMBER	8000	5	8
CMD	CMD	6	1
LBRACKET	[	6	5
STRING	"python"	6	6
COMMA	,	6	14
STRING	"app.py"	6	16
RBRACKET	]	6	24

```
-----
Resultado: archivo válido, sin errores léxicos.
```

Implementación de un Lexer para un subconjunto de Rust usando Flex

Una gramática es **ambigua** cuando una misma cadena puede derivarse de más de una forma, produciendo diferentes árboles sintácticos. Esto representa un problema para el compilador, ya que una misma expresión podría interpretarse con distintos significados.

Manual de Usuario del Metacompilador Flex

Flex (Fast Lexical Analyzer Generator) es un metacompilador utilizado para generar analizadores léxicos de forma automática. Su funcionamiento se basa en la definición de reglas mediante expresiones regulares, donde cada patrón reconocido ejecuta una acción previamente definida por el programador.

El archivo de especificación de Flex posee la extensión **.l** y se divide en tres secciones principales:

- **Definiciones:** se incluyen librerías, variables y configuraciones generales.
- **Reglas:** contiene las expresiones regulares y las acciones que ejecutará el analizador al reconocer un token.
- **Código auxiliar:** incluye funciones adicionales, como el programa principal (**main**).

Una vez creado el archivo **.l**, Flex genera automáticamente un archivo en lenguaje C denominado **lex.yy.c**, el cual puede compilarse utilizando un compilador como GCC para obtener el analizador léxico ejecutable.

El procedimiento general para utilizar Flex es el siguiente:

1. Crear el archivo de especificación (**lexer.l**).
2. Generar el código fuente mediante Flex.
3. Compilar el código generado.

4. Ejecutar el analizador indicando el archivo que se desea analizar.
5. Los comandos utilizados son:

```
flex lexer.l
```

```
gcc lex.yy.c -o lexer
```

```
./lexer programa.rs
```

Descripción del lenguaje L

Para esta actividad se diseñó un lenguaje **L**, correspondiente a un subconjunto simplificado del lenguaje de programación **Rust**.

El objetivo del lenguaje no es representar todas las características de Rust, sino únicamente los elementos necesarios para demostrar el funcionamiento del analizador léxico.

Los componentes reconocidos por el lenguaje son:

- Declaración de variables mediante la palabra reservada **let**.
- Definición de funciones utilizando **fn**.
- Estructuras condicionales mediante **if**.
- Ciclos mediante **while**.
- Sentencia **return**.
- Identificadores.
- Números enteros.
- Operadores aritméticos.
- Operadores de asignación.
- Llaves.
- Paréntesis.
- Punto y coma.

Los principales tokens definidos para el lenguaje se presentan en la siguiente tabla.

Tabla 4

Los principales tokens definidos para el lenguaje se presentan en la siguiente tabla.

Token	Descripción
LET	Declaración de variable
FN	Declaración de función
IF	Estructura condicional
WHILE	Ciclo
RETURN	Retorno de función
IDENTIFIER	Identificador
NUMBER	Número entero
ASSIGN	Operador =
OPERATOR	Operadores + - * /
LBRACE	Llave izquierda
RBRACE	Llave derecha
LPAREN	Paréntesis izquierdo
RPAREN	Paréntesis derecho
SEMICOLON	Punto y coma

Nota. El analizador léxico fue desarrollado utilizando Flex. Para ello, cada token del lenguaje fue representado mediante una expresión regular y asociado a una acción encargada de mostrar el token reconocido y su lexema correspondiente.

Implementación

El analizador léxico fue desarrollado utilizando Flex. Para ello, cada token del lenguaje fue representado mediante una expresión regular y asociado a una acción encargada de mostrar el token reconocido y su lexema correspondiente.

Durante la ejecución, el programa lee el archivo fuente escrito en el lenguaje L y analiza cada uno de sus caracteres hasta reconocer los diferentes componentes léxicos definidos previamente.

Cuando un elemento coincide con alguna de las expresiones regulares, imprime el nombre del token junto con el lexema encontrado. Si no, el programa informa la existencia de un error léxico.

Instalación y ejecución

Para generar el analizador léxico es necesario disponer de Flex y GCC instalados. Una vez creado el archivo `lexer.l`, se ejecutan los siguientes comandos:

```
flex lexer.l
```

```
gcc lex.yy.c -o lexer
```

```
./lexer programa.rs
```

Como archivo de prueba se utilizó el siguiente programa:

```
fn suma() {
```

```
let x = 10;
```

```
if (x > 5) {
```

```
return;
```

```
}  
  
}
```

La salida del programa muestra cada uno de los tokens identificados durante el análisis léxico.

Figura 1

Salida del analizador léxico para el primer ejemplo de Dockerfile, donde se identifican correctamente los tokens y lexemas reconocidos durante el proceso de tokenización.

```
danil@LAPTOP-ESQJE27K:/mnt/c/Users/danil/Documents/Dev/uni/lc/tema4$ ./lexer programa.rs  
TOKEN: FN, Lexema: fn  
TOKEN: IDENTIFIER, Lexema: suma  
TOKEN: LPAREN, Lexema: (  
TOKEN: RPAREN, Lexema: )  
TOKEN: LBRACE, Lexema: {  
TOKEN: LET, Lexema: let  
TOKEN: IDENTIFIER, Lexema: x  
TOKEN: ASSIGN, Lexema: =  
TOKEN: NUMBER, Lexema: 10  
TOKEN: SEMICOLON, Lexema: ;  
TOKEN: IF, Lexema: if  
TOKEN: LPAREN, Lexema: (  
TOKEN: IDENTIFIER, Lexema: x  
TOKEN: GREATER_THAN, Lexema: >  
TOKEN: NUMBER, Lexema: 5  
TOKEN: RPAREN, Lexema: )  
TOKEN: LBRACE, Lexema: {  
TOKEN: RETURN, Lexema: return  
TOKEN: SEMICOLON, Lexema: ;  
TOKEN: RBRACE, Lexema: }  
TOKEN: RBRACE, Lexema: }
```

Aplicación de Flex en Seguridad Informática.

En el área de la seguridad informática existen diferentes lenguajes especializados para la detección de amenazas, análisis de archivos y creación de reglas de inspección. Uno de los más utilizados es el lenguaje **YARA**, empleado para identificar malware mediante reglas basadas en patrones.

Un analizador léxico construido con Flex puede utilizarse para reconocer los diferentes componentes que conforman una regla YARA antes de realizar su análisis sintáctico.

Considere el siguiente ejemplo:

```
rule Malware {
```

strings:

\$a = "malware"

condition:

\$a

}

Tabla 5

La tokenización correspondiente se presentan en la siguiente tabla.

Token	Lexema
RULE	rule
IDENTIFIER	Malware
LBRACE	{
STRINGS	strings
IDENTIFIER	\$a
ASSIGN	=
STRING	"malware"
CONDITION	condition

IDENTIFIER

\$a

RBRACE

}

Nota. Este tipo de análisis permite verificar que las reglas estén correctamente escritas antes de ser utilizadas por herramientas de detección, reduciendo errores durante su procesamiento.

Conclusiones

El desarrollo de esta actividad permitió comprender la importancia del análisis léxico como primera etapa del proceso de compilación. Mediante el estudio de los autómatas de pila se identificó su utilidad para el reconocimiento de lenguajes libres de contexto y su diferencia respecto a los autómatas finitos.

Asimismo, la implementación de un analizador léxico mediante expresiones regulares permitió comprender cómo se reconocen los componentes léxicos de un lenguaje de forma manual, mientras que el uso de Flex demostró que es posible automatizar este proceso, facilitando el desarrollo de analizadores más robustos y reduciendo el esfuerzo de implementación.

Finalmente, se observó que los analizadores léxicos no solo son utilizados en compiladores, sino también en otras áreas de la informática, como la seguridad informática, donde contribuyen al procesamiento y validación de lenguajes especializados.

Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson.

Docker Inc. (2024). *Dockerfile reference*. <https://docs.docker.com/reference/dockerfile/>

Flex Project. (2024). *Flex: The fast lexical analyzer generator*. <https://westes.github.io/flex/manual/>

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3rd ed.). Pearson.

The Rust Project Developers. (2024). *The Rust programming language*. <https://doc.rust-lang.org/book/>

The Rust Project Developers. (2024). *Rust reference*. <https://doc.rust-lang.org/reference/>

YARA Project. (2024). *YARA documentation*. <https://yara.readthedocs.io/>

